

Agentic SDD

1. Product Vision

Agentic SDD is a lightweight, repository-based framework for **Spec-Driven Development with AI agent teams**.

Its goal is to make an AI coding agent behave as part of a structured software development team, while keeping the repository itself as the source of truth for:

- product intent
- specifications
- backlog
- sprint history
- agent roles
- workflow and governance

The developer should primarily interact with the system through the **AI conversation inside their IDE**, rather than through a project-management UI or a complex CLI.

The framework must work across AI coding environments such as Claude Code and Cursor, while avoiding vendor lock-in.

The framework should be opinionated enough to provide a complete development methodology, but simple enough that a development team can modify the agents and configuration for its own needs.

2. Core Development Model

The product follows this lifecycle:

```
Product Vision
  ↓
Specs
  ↓
Backlog
  ↓
Sprint
  ↓
Agentic Development
  ↓
Agent Reviews
  ↓
Human Review
  ↓
Approved
  ↓
```

Done



Release / Deployment

There are intentionally **no mandatory intermediate layers for Requirements, Capabilities, Features, Tasks or User Stories**.

A specification is the durable representation of what the product or system must do.

User Stories may be generated or used by an agent when useful for planning, but they are not required to be persistent artefacts in the methodology.

The objective is to avoid reproducing traditional Agile bureaucracy for AI development.

3. Repository as Source of Truth

After initialization, an Agentic SDD repository should have a structure similar to:

```
/
├── product/
│   └── vision.md
├── specs/
├── backlog/
│   └── backlog.md
├── sprints/
├── agents/
├── AGENTS.md
└── .sdd/
    └── config.yaml
```

The exact structure should be configurable in the future, but the initial implementation should use the structure above.

Product Vision

product/vision.md

Contains the long-lived product context:

- product purpose

- problems being solved
- target users
- product principles
- strategic goals
- important constraints
- explicit out-of-scope areas where relevant

The Product Vision is intentionally high-level.

It must not become a technical design document.

4. Specifications

Specifications live under:

specs/

They represent the durable functional and/or technical intent of the system.

A specification can describe:

- new functionality
- changes to existing functionality
- business rules
- technical requirements
- architectural requirements
- security requirements
- infrastructure requirements

A specification should normally contain:

- objective
- context
- expected behaviour
- constraints
- acceptance criteria
- relevant edge cases
- dependencies
- open questions, if any

Specifications should describe **what must be true**, rather than prescribing implementation unnecessarily.

For example, a specification should not normally dictate:

Implement this using Django service X.

unless that is actually a project constraint.

5. Product-to-Spec Cascade

The system must support the concept of changes propagating downwards.

For example:

```
product/vision.md
      ↓
affected specifications
      ↓
backlog
      ↓
future sprint
```

If a developer tells the Product Agent:

We now want customers to support multiple payment providers.

The agent should inspect the existing Vision and Specs and identify affected specifications.

It should not silently modify everything.

Instead it should be able to report:

```
Vision change detected.

Affected specifications:
- payment-processing
- payment-configuration

Potentially affected:
- reporting

No impact detected:
- authentication

Human approval required before modifying specifications.
```

This provides traceability without requiring a separate requirements-management system.

6. Backlog

The backlog is derived from approved specifications.

The backlog should contain development units that can be planned into a sprint.

A backlog item should reference its source specification.

Example:

```
PAY-001
Stripe Connect onboarding

Source:
specs/payments/stripe-connect.md
```

The Planner Agent is responsible for turning specifications into backlog items and estimating their effort.

The system should not require a specific estimation methodology initially.

The Planner should be able to determine:

- approximate effort
- dependencies
- whether an item fits into a sprint
- whether one or more development teams are appropriate

7. Sprint Model

A sprint represents an execution period and its persistent engineering context.

Example:

```
sprints/
├─ sprint-001/
│  ├─ sprint.md
│  ├─ decisions.md
│  ├─ feedback.md
│  ├─ review.md
│  └─ status.md
```

The sprint should preserve relevant context generated during implementation.

This includes:

- decisions
- architectural decisions
- human feedback
- changes requested during review

- blockers
- important implementation discoveries
- relevant deviations from the original specification

The sprint should **not** become a dump of every agent conversation or execution log.

Only useful engineering/product context should be persisted.

This allows agents to recover the context of previous work without relying on a long-running conversation.

8. AI Agent Team

The framework provides a default multidisciplinary agent team.

The initial team should include roles such as:

```
Product  
Planner  
Engineering Manager  
Architect  
Engineer  
UX  
QA  
Security  
Platform  
Release
```

The exact role names must be configurable.

For example:

```
agents:  
  product: product  
  planner: planner  
  engineering_manager: engineering-manager  
  architect: architect  
  engineer: engineer  
  ux: ux  
  qa: qa  
  security: security  
  platform: platform  
  release: release
```

A project must be able to rename an agent without changing the methodology.

For example:

```
agents:
  architect: solution-architect
```

The framework should therefore refer to agents by **logical role**, while `config.yaml` maps that role to an actual agent definition.

9. Agent Definitions

Agent definitions must be written using an established, portable agent-instruction standard where one is available and appropriate.

The implementation must investigate the current standards supported by Claude Code, Cursor and other major agentic development environments and select the most interoperable approach.

The framework should avoid creating a proprietary agent format unless there is no suitable existing standard.

The important principle is:

```
Agentic SDD methodology
    ↓
standard agent definition
    ↓
tool-specific adapter
    ↓
Claude / Cursor / other IDE
```

The actual agent files should live in the repository:

```
agents/
```

They belong to the project after initialization and may be modified by the development team.

The framework must not depend on agents remaining inside the installed package.

10. Agent Responsibilities

Each agent must have a clearly defined responsibility.

For example:

Product Agent

Responsible for:

- interpreting changes to the Product Vision
- identifying affected specifications
- creating or modifying specifications
- identifying ambiguities
- protecting product intent

It must not implement application code.

Planner Agent

Responsible for:

- analysing approved specifications
- creating backlog items
- estimating effort
- identifying dependencies
- proposing sprint scope

Engineering Manager Agent

Responsible for:

- coordinating the development team
- monitoring sprint scope
- identifying blockers
- identifying excessive complexity
- ensuring appropriate reviews happen
- providing an objective engineering assessment

Engineer Agent

Responsible for implementation.

UX Agent

Responsible for:

- UX flows
- interaction design
- usability considerations
- UI-related decisions

QA Agent

Responsible for:

- test strategy
- automated tests

- acceptance verification
- regression detection

Security Agent

Responsible for:

- security review
- security requirements
- vulnerability detection
- security-related implementation review

Platform Agent

Responsible for:

- infrastructure
- CI/CD
- deployment
- observability
- operational concerns

Release Agent

Responsible for:

- release preparation
- release notes
- deployment readiness
- rollback considerations

The roles should remain independently configurable.

11. Human-in-the-Loop

Human approval is a fundamental part of the methodology.

Agents can:

- create specifications
- modify code
- create backlog items
- implement features
- run tests
- perform reviews

But agents must not autonomously declare a feature permanently when human approval is required.

The workflow should support states such as:

```
draft
approved
planned
in_progress
ready_for_review
changes_requested
approved
done
```

The exact state model should be defined centrally by the framework.

12. Human Feedback Loop

After implementation, the work enters human review.

The developer can provide feedback directly through the IDE conversation.

The relevant feedback must be persisted in the sprint:

```
sprints/sprint-001/feedback.md
```

Example:

```
## PAY-001

Status: changes_requested

Feedback:

The onboarding flow must support reconnecting
an expired Stripe authorization.

Approved:
No
```

The development agents then continue working from that feedback.

The cycle is:

```
Implementation
  ↓
Review
  ↓
Human feedback
```

```
    ↓
Changes
    ↓
Review
    ↓
Human approval
    ↓
Done
```

This loop can occur multiple times within the same sprint.

13. Agent Reviews

Agent reviews should be independent from implementation where possible.

For example:

```
Implementation
|
|— QA Review
|— Security Review
|— Architecture Review
|— Engineering Manager Review
|
▼
Human Review
```

Not every review must run for every change.

The workflow configuration should determine when reviews are mandatory or conditional.

For example:

```
reviews:
  security: required
  qa: required
  architecture: conditional
```

Security review should be capable of running both:

- during development
- as part of CI/CD

14. CI/CD Governance

The framework should be designed so that additional agents can later be integrated into CI/CD.

Examples:

```
Pull Request
  ↓
Security Auditor
  ↓
QA / Test Auditor
  ↓
Architecture / Engineering Review
  ↓
Human approval
  ↓
Merge
```

The initial product does not need to implement all CI/CD integrations.

The agent definitions and workflow should, however, be designed to support them.

15. AGENTS.md

`AGENTS.md` should be intentionally small.

It should contain only the essential project-wide instructions and point agents towards the SDD structure.

It should not contain the complete methodology or every agent's instructions.

Example responsibilities:

- This repository follows Agentic SDD.
- Product intent is in product/vision.md.
- Specifications are in specs/.
- Backlog is in backlog/.
- Sprint context is in sprints/.
- Agent definitions are in agents/.
- Human approval is required before work is considered done.

The detailed methodology should live in dedicated framework files and the individual agent definitions.

16. Existing AGENTS.md

`agentic-sdd init` must detect whether an `AGENTS.md` already exists.

It must **never silently overwrite it**.

If an existing file is detected, the installer should integrate the SDD instructions safely.

Where supported by the target agent environment, the preferred approach is to use an inclusion/reference mechanism such as:

```
@...
```

rather than duplicating the same instructions.

The goal is:

```
Existing project instructions
      +
Agentic SDD instructions
```

without creating two competing sources of truth.

If the target environment does not support the required inclusion mechanism, `init` must use the safest compatible alternative and inform the developer.

17. Claude, Cursor and Tool Compatibility

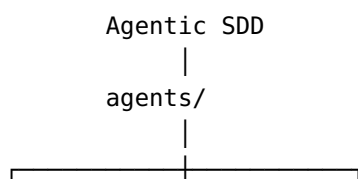
Agentic SDD must not make Claude Code the canonical representation of the methodology.

The repository should contain a common source of truth:

```
agents/
```

Tool-specific configuration should be generated or linked where required.

Conceptually:



▼
Claude
adapter

▼
Cursor
adapter

▼
Other
adapter

The implementation must verify the current supported conventions of each target tool rather than assuming that the same file or directory is automatically loaded by every IDE.

18. Configuration

The framework should have a small configuration file:

```
.sdd/config.yaml
```

This file is intended to configure the methodology, not the application itself.

Example:

```
version: 1

agents:
  product: product
  planner: planner
  engineering_manager: engineering-manager
  architect: architect
  engineer: engineer
  ux: ux
  qa: qa
  security: security
  platform: platform
  release: release

workflow:
  human_approval: required

reviews:
  security: required
  qa: required
  architecture: conditional
```

The configuration must support renaming or replacing agents.

For example:

```
agents:  
  architect: solution-architect
```

The framework must use logical roles internally rather than hard-coding filenames.

19. CLI

The initial CLI should remain intentionally small.

Only two commands are required:

```
agentic-sdd init  
agentic-sdd status
```

agentic-sdd init

Initializes an existing repository with the Agentic SDD structure.

It should:

1. Detect the repository.
2. Detect existing agent configuration.
3. Detect existing `AGENTS.md`.
4. Create the SDD directories and initial files.
5. Install the default agent definitions.
6. Create `.sdd/config.yaml`.
7. Configure the appropriate IDE/agent integration.
8. Never silently overwrite existing project instructions.

There should be **no development profile selector** in the initial version.

There should be **no template system** in the initial version.

The framework itself defines the initial methodology and default agent team.

agentic-sdd status

Provides a deterministic overview of the current SDD state.

For example:

```
Agentic SDD
```

```
Product
```

Vision: OK

Specs

Total: 14

Draft: 2

Approved: 12

Backlog

Items: 21

Sprint

Current: sprint-004

Completed: 5/8

Governance

Pending human review: 1

Spec inconsistencies: 0

`status` should primarily inspect repository state.

It should not require an LLM to perform deterministic checks.

20. No `update` command

The framework should not implement its own framework-update mechanism initially.

The package manager is responsible for updating the Agentic SDD package.

For example:

```
pip / uv / npm / etc.
```

However, the framework must be designed carefully so that updating the package does not unexpectedly overwrite project-owned agent definitions or project customizations.

21. Installation Model

Agentic SDD should be distributed as a normal installable developer tool.

The developer should be able to install it and run:

```
agentic-sdd init
```

inside any existing repository.

The package provides:

- CLI
- default agent definitions
- workflow definitions
- schemas
- IDE adapters
- validation logic
- initialization logic

After initialization, the repository contains the relevant SDD artefacts locally.

The repository should remain functional even if the Agentic SDD package is subsequently unavailable.

This is important for portability and long-term maintainability.

22. No Mandatory MCP in Version 1

MCP should not be required for the first version.

The repository itself provides the primary context:

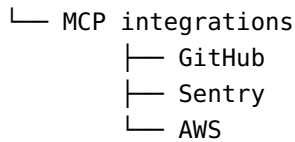
```
Vision
Specs
Backlog
Sprints
Agents
Code
```

MCP may be added later as an integration layer for external systems such as:

- GitHub
- Sentry
- AWS
- Linear
- Jira
- Datadog
- other engineering platforms

The conceptual architecture should allow this:

```
Agent
|
|— Repository
|
```



MCP must remain an integration mechanism, not the source of truth for the SDD methodology.

23. Design Principles

The implementation must follow these principles.

Conversation first

The developer should primarily interact with Agentic SDD through the AI agent conversation in their IDE.

Repository first

The repository is the durable source of truth.

Agents are configurable

The default team is opinionated but replaceable.

Human authority

Agents can propose and implement. Humans approve.

Minimal bureaucracy

Do not reproduce traditional Agile artefacts unless they provide real value to an AI development team.

Tool agnostic

Do not make the methodology dependent on Claude, Cursor or a particular model.

Deterministic where possible

Use code for validation and state management.

Use agents for reasoning and decision-making.

Persistent context

Sprint history should preserve important decisions and feedback so that future agents can recover context without relying on conversation history.

Standards over proprietary formats

Where an established agent-definition or IDE-integration standard exists, use it instead of inventing a proprietary one.

24. Example Developer Experience

A developer starts with an existing repository:

```
cd my-project  
agentic-sdd init
```

The framework creates the SDD structure.

The developer then opens Cursor or Claude Code.

They write:

I want customers to be able to connect their Stripe account and receive payments from their own customers.

The Product Agent reads:

```
product/vision.md  
specs/
```

and determines whether this is:

- a new specification
- a modification to an existing specification
- or something already covered

It creates or proposes the relevant specification.

The developer reviews and approves it.

The Planner Agent turns the approved specification into backlog items.

The Engineering Manager proposes sprint scope.

The development team of agents implements the work.

QA, Security and Architecture agents review where appropriate.

The developer reviews the result.

If changes are required, the feedback is persisted into:

```
sprints/sprint-N/feedback.md
```

The agents continue iterating.

Once the developer approves the work:

```
backlog item → done
```

The sprint retains the relevant implementation history and decisions.

The process then continues with the next backlog item.

The developer does not need to manually orchestrate each agent through a CLI.

The conversation is the primary interface.

25. Initial Scope

Version 1 should focus on doing a small number of things extremely well:

1. Initialize a repository with Agentic SDD.
2. Provide a strong default multidisciplinary agent team.
3. Provide portable agent definitions.
4. Provide a clean Product Vision → Specs → Backlog → Sprint workflow.
5. Persist sprint context and human feedback.
6. Support configurable agent names/roles.
7. Integrate safely with an existing `AGENTS.md`.
8. Provide compatibility with Claude Code and Cursor.
9. Provide deterministic `status` and repository validation.
10. Keep the framework independent from any specific LLM.

The following should explicitly remain outside Version 1:

- project-management UI
- Jira/Linear replacement
- MCP integrations
- autonomous production deployment
- multiple development profiles
- template marketplace
- complex CLI workflow commands
- proprietary agent protocol
- automatic uncontrolled propagation of Product Vision changes
- mandatory User Story artefacts
- mandatory Requirements or Capabilities layers